

Performance Testing

Date	30 June 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter 5.6.3
Type of Testing	Load Testing
Target Module/API	Flask Web Application - Loan Prediction Module (/predict)
Test Environment	Localhost (Python Flask Server)
Test Date	30 June 2026

Step2: Test Scenarios

S.No	Test Scenario/ Description	No.of Virtual Users	Duration(sec)	Expected Outcome
1	Normal load testing with multiple users accessing the loan prediction application	10	30	Application should respond successfully without errors
2	Medium load testing with multiple prediction requests	50	60	Application should handle requests with low response time
3	High load testing to check application stability	100	60	Application should remain stable with acceptable performance
4	Continuous prediction request testing	50	120	Prediction service should work consistently without failures

Step3:Performance Test Results

S.No	Metric	Target Value	Actual Value	Status(Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	59 ms	Pass	Fast response from Flask prediction service
2	Response Time (Max)	< 5 seconds	199 ms	Pass	Maximum response time is within acceptable limit
3	Throughput (Req/sec)	High request handling	47.6 requests/sec	Pass	Application successfully handled multiple requests.
4	Error Rate	< 1%	0%	Pass	No failed requests during testing
5	CPU Utilization	< 80%	2%	Pass	System operated normally during testing
6	Memory Utilization	< 80%	1.45%	Pass	No memory issues observed

Step4: Observations & Analysis:

Key Findings:

The Smart Lender application successfully handled 500 requests with **0% errors**. The average response time was **59 ms**, showing fast loan prediction performance.

Bottlenecks Identified:

No major bottlenecks were observed during testing. The application performed smoothly under the tested load.

Optimization Steps Taken:

- Saved the trained model and scaler for faster predictions.
- Used modular Flask structure for better performance.
- Applied preprocessing before prediction to improve efficiency.

Step5:Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

View Results Tree.jmx (C:\Users\HP\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin\View Results Tree.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:10 0 0/50

Test Plan

Thread

HT

Sur

View

HT

Sur

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	500	59	19	199	33.31	0.00%	47.6/sec	147.03	18.49	3165.0
TOTAL	500	59	19	199	33.31	0.00%	47.6/sec	147.03	18.49	3165.0

☐ Include group name in label? ☒ Save Table Header

View Results Tree.jmx (C:\Users\HP\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin\View Results Tree.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:10 0 0/50

Test Plan

Thread Group

HTTP Requ

Summary R

View Result

HTTP Requ

Summary R

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	550	57	19	199	33.18	0.00%	43.0/min	2.21	0.28	3165.0
TOTAL	550	57	19	199	33.18	0.00%	43.0/min	2.21	0.28	3165.0

☐ Include group name in label? ☒ Save Table Header